mini-DM — instrukcja wdrożenia na serwer demo

Wersja aplikacji	commit `69b80f5` (gałąź `main`)
Poprzednio wdrożona wersja	commit `d1b958a` (wdrożenie z 2026-11-08)
Data przygotowania	2026-11-08
Środowisko	serwer demo klienta (`<ADRES_DEMO>`), Docker + docker compose, reverse proxy nginx

0. Konto i uprawnienia

Całe wdrożenie wykonuj ze zwykłego konta administratora serwera, `<UZYTKOWNIK>` (zob. punkt 2): przesyłanie plików z Windows przez WinSCP, logowanie przez SSH i polecenia na serwerze. **Nie loguj się jako root.** Wtedy pomyłka w poleceniu `rm` lub `unzip` w złym katalogu nie ma pełnych uprawnień.

- `<UZYTKOWNIK>` musi należeć do grupy `docker` (sprawdź poleceniem `id`). Wtedy `docker build`, `docker tag`, `docker run` i `docker compose` działają **bez `sudo`**, co potwierdzi polecenie `docker ps`. Jeśli zgłasza `permission denied`, wyloguj się i zaloguj ponownie, bo członkostwo w grupie działa dopiero w nowej sesji.
- Konto, z którego uruchamiasz `docker build`, nie ma wpływu na obraz. Obraz i tak buduje demon Dockera, a właściciela plików w obrazie ustala `Dockerfile` (użytkownik `app`).
- `<KATALOG_ZRODEL>` i `<KATALOG_KOPII>` powinny należeć do `<UZYTKOWNIK>`, np. `~/mini-dm-src` i `~/kopie-mini-dm`.
- **`sudo` jest potrzebne tylko** przy operacjach na plikach bazy w `<KATALOG_DANYCH>`, jeśli nie należą one do `<UZYTKOWNIK>`. Dotyczy to `chmod` w kroku 6 oraz przywracania bazy w kroku 9. W tych miejscach polecenia mają już `sudo`.

1. Co zawiera to wdrożenie

Zmiany względem `d1b958a`:

1. **Dymek z powodem oznaczenia „wymaga uwagi".** Po najechaniu kursorem na żółty trójkąt na liście zamówień lub w raporcie pojawia się opis, np. „Wymaga uwagi: Dostawa zakończona, brak wyniku badań". Gdy pasuje kilka reguł, dymek wymienia wszystkie:
   - „Oznaczone ręcznie jako ważne",
   - „ETA cel w ciągu 3 dni lub minęła, brak daty dostawy",
   - „Dostawa zakończona, brak wyniku badań".
2. Same reguły oznaczania się **nie zmieniają**. Filtr „Tylko wymagające uwagi" i kafelek Dashboardu pokazują te same liczby co przed wdrożeniem.

Dymek wyjaśnia sytuację zgłoszoną po poprzednim wdrożeniu: po odznaczeniu „Oznacz jako ważne / wymagające uwagi" zamówienie może nadal być podświetlone, bo spełnia regułę automatyczną.

To wdrożenie nie zmienia struktury bazy, bo nie zawiera nowej migracji. Nie zmieniają się też zmienne środowiskowe, port (4321) ani konfiguracja `docker-compose.yml` i nginx.

> ***Jeśli serwer ma wersję starszą niż `d1b958a`**, to wdrożenie zawiera też filtr i kafelek „Brak badań dostawcy" (`d1b958a`). Jeśli jest starsza niż `95062ff` (sprzed migracji nr 4), zawiera ponadto:*

> - *pole „Badania laboratoryjne od dostawcy”,*
> - *przeniesione pole „Zablokowane”,*
> - *poszerzony layout,*
> - ***migrację nr 4, która przebudowuje tabelę*** `purchase_orders` .
>
> *W takim przypadku kopia zapasowa bazy (krok 4) jest absolutnie obowiązkowa. Różnice w oczekiwanych wynikach opisują uwagi „_starsza wersja_” w krokach 4.4, 8 i 9.*

2. Oznaczenia używane w instrukcji

Symbol	Znaczenie
`<UZYTKOWNIK>`	zwykłe (nie-root) konto na serwerze, w grupie `docker`, używane też w WinSCP
`<ADRES_DEMO>`	domena, pod którą działa serwer demo
`<KATALOG_ZRODEL>`	katalog na serwerze, w którym budowany jest obraz (kopia repozytorium)
`<KATALOG_COMPOSE>`	katalog z plikiem `docker-compose.yml`, w którym zdefiniowana jest usługa `mini-dm`
`<KATALOG_DANYCH>`	katalog z bazą SQLite, podmontowany w kontenerze jako `/app/data` (bind mount z `docker-compose.yml`)
`<KATALOG_KOPII>`	dowolny katalog na kopie zapasowe, poza `<KATALOG_DANYCH>`

Rzeczywiste wartości tych oznaczeń celowo nie są zapisane w repozytorium. Ścieżki odczytasz z `docker-compose.yml` na serwerze.

Instrukcja zakłada, że usługa `mini-dm` w `docker-compose.yml` korzysta z obrazu `mini-dm:latest` , tak jak przy poprzednich wdrożeniach.

3. Przygotowanie paczki ze źródłami (lokalnie)

Na komputerze deweloperskim, w katalogu projektu:

```
git archive --format=zip -o mini-dm-69b80f5.zip 69b80f5
```

`git archive` pakuje tylko pliki śledzone przez git. W paczce nie ma więc `node_modules` , `dist` , `.astro` , `data` , `.env*` ani lokalnych konfiguracji.

Skopiuj `mini-dm-69b80f5.zip` na serwer przez WinSCP, logując się jako `<UZYTKOWNIK>` , do katalogu domowego (`~`). Następnie wyczyść `<KATALOG_ZRODEL>` ze starych plików źródłowych i rozpakuj tam paczkę:

```
cd <KATALOG_ZRODEL>
unzip -o ~/mini-dm-69b80f5.zip
```

> *Nigdy nie rozpakowuj paczki do `<KATALOG_DANYCH>` i nie czyść tego katalogu.*

4. Zatrzymanie aplikacji i kopia zapasowa bazy

4.1. Zachowaj obecny obraz pod osobną nazwą, żeby można było do niego wrócić. Jeśli `mini-dm:prev` został po poprzednim wdrożeniu, zostanie nadpisany, i tak ma być.

```
docker tag mini-dm:latest mini-dm:prev
```

4.2. Zatrzymaj aplikację. SQLite pozwala na jednego zapisującego, a kopia musi być spójna:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm
```

4.3. Zrób kopię **wszystkich** plików bazy, czyli `mini-dm.db` oraz, jeśli istnieją, `mini-dm.db-wal` i `mini-dm.db-shm` :

```
STAMP=$(date +%Y%m%d-%H%M)
mkdir -p <KATALOG_KOPII>/mini-dm-$STAMP
cp -a <KATALOG_DANYCH>/mini-dm.db* <KATALOG_KOPII>/mini-dm-$STAMP/
ls -l <KATALOG_KOPII>/mini-dm-$STAMP/
```

4.4. Zanotuj stan bazy przed wdrożeniem. Polecenie uruchamia na chwilę stary obraz w trybie tylko do odczytu:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:prev node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
'
```

Zapisz liczby `zamowienia` i `historia` . Pole `migracje` powinno pokazać `1,2,3,4` .

Starsza wersja: jeśli `migracje` pokazuje `1,2,3` , serwer ma wersję sprzed migracji nr 4 (zob. uwagę w punkcie 1).

5. Budowa nowego obrazu

```
cd <KATALOG_ZRODEL>
docker build -t mini-dm:latest .
```

Budowa powinna zakończyć się bez błędów. Etap `npm run build` jest w niej wymagany.

6. Kontrola uprawnień do plików bazy

Kontener działa jako nieuprzywilejowany użytkownik `app` . Przy jednym z wcześniejszych wdrożeń pliki bazy należały do innego użytkownika z prawami `644` . Aplikacja wtedy startowała i wyświetlała dane, ale **pierwszy zapis kończył się błędem**.

```
ls -l <KATALOG_DANYCH>/
```

Jeśli pliki `mini-dm.db*` nie mają praw zapisu dla „others” (`-rw-rw-rw-`), wykonaj:

```
sudo chmod 666 <KATALOG_DANYCH>/mini-dm.db*
```

Sam katalog również musi być zapisywalny dla kontenera (wcześniej miał prawa `777`).

7. Uruchomienie nowej wersji

```
cd <KATALOG_COMPOSE>
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

W logach **nie** powinno być błędów typu `Migration ...`, `SQLITE_...` ani `foreign key violation`. Migracje nie wypisują komunikatu o sukcesie, więc brak błędów i działający serwer oznaczają, że start się udał.

8. Weryfikacja

8.1. Baza. Uruchom to samo sprawdzenie co w kroku 4.4, tym razem na nowym obrazie:

```
docker run --rm -v <KATALOG_DANYCH>:/app/data mini-dm:latest node -e '
const db = require("better-sqlite3")("/app/data/mini-dm.db", { readonly: true,
fileMustExist: true });
console.log("zamowienia:", db.prepare("SELECT COUNT(*) AS c FROM purchase_orders").get().c);
console.log("historia:  ", db.prepare("SELECT COUNT(*) AS c FROM
purchase_order_history").get().c);
console.log("migracje:  ", db.prepare("SELECT version FROM schema_migrations ORDER BY
version").all().map(r => r.version).join(","));
'
```

Oczekiwany wynik:

- `zamowienia` i `historia` mają **te same liczby** co w kroku 4.4,
- `migracje` pokazuje `1,2,3,4`.

Starsza wersja: `migracje` zmienia się z `1,2,3` na `1,2,3,4`, a liczby zamówień i historii pozostają te same.

8.2. Aplikacja w przeglądarce (`https://<ADRES_DEMO>`):

1. Zaloguj się. Kafelek „Wymagające uwagi" na **Dashboardzie** pokazuje tę samą liczbę co przed wdrożeniem.
2. Na liście **Zamówień** najedź kursorem na żółty trójkąt przy kilku podświetlonych zamówieniach. Kursor zmienia się w znak zapytania, a dymek zaczyna się od „Wymaga uwagi:" i podaje co najmniej jeden powód. To samo sprawdź w zakładce **Raporty**.
3. **Test zapisu (obowiązkowy):** otwórz edycję zamówienia, które **nie** jest podświetlone, zaznacz „Oznacz jako ważne / wymagające uwagi" i zapisz.
4. Na liście zamówienie jest teraz podświetlone, a dymek nad trójkątem pokazuje „Wymaga uwagi: Oznaczone ręcznie jako ważne". Liczba na kafelku „Wymagające uwagi" zwiększyła się o 1.
5. Ponownie otwórz edycję tego zamówienia. W historii zmian powinien być wpis „Ważne: Nie → Tak". Odznacz pole i zapisz, żeby przywrócić poprzedni stan. Podświetlenie i trójkąt znikają.

Wdrożenie jest zakończone dopiero po udanym teście zapisu (punkty 3–5), a nie po samym wyświetleniu strony.

9. Wycofanie zmian (rollback)

Wykonaj te kroki tylko wtedy, gdy krok 7 lub 8 się nie powiedzie.

To wdrożenie nie zmienia struktury bazy, więc zwykle wystarczy **powrót do poprzedniego obrazu**, a dane wprowadzone w międzyczasie zostają zachowane:

```
cd <KATALOG_COMPOSE>
docker tag mini-dm:prev mini-dm:latest
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

Bazę z kopii przywracaj tylko wtedy, gdy jest uszkodzona albo serwer miał *starszą wersję* (`migracje: 1,2,3` w kroku 4.4). Dane wprowadzone po kroku 7 zostaną wtedy utracone. Przywróć bazę **razem** z obrazem:

```
cd <KATALOG_COMPOSE>
docker compose stop mini-dm

# usuń bieżące pliki -wal/-shm, żeby nie zostały po nowej wersji
sudo rm -f <KATALOG_DANYCH>/mini-dm.db-wal <KATALOG_DANYCH>/mini-dm.db-shm
sudo cp -a <KATALOG_KOPII>/mini-dm-<STAMP>/mini-dm.db* <KATALOG_DANYCH>/
sudo chmod 666 <KATALOG_DANYCH>/mini-dm.db*

docker tag mini-dm:prev mini-dm:latest
docker compose up -d mini-dm
docker compose logs --tail 50 mini-dm
```

Po rollbacku powtórz sprawdzenie z kroku 4.4 na obrazie `mini-dm:latest`. Liczby i lista migracji powinny być takie same jak przed wdrożeniem.

10. Po udanym wdrożeniu

- Zostaw kopię z kroku 4.3 i obraz `mini-dm:prev` przez kilka dni. Gdy klient potwierdzi, że wszystko działa, możesz usunąć obraz poleceniem `docker image rm mini-dm:prev`.
- Usuń paczkę `mini-dm-69b80f5.zip` z serwera.